

FULL STACK WEB DEVELOPMENT

Name: TAMMINA USHASRI

Roll No: A24126552120

Branch: CSM

Section: B

Subject: FS

Experiment Title: To-Do List Application Using JavaScript DOM Manipulation

1. OBJECTIVE / AIM

To develop an interactive To-Do List application using HTML, CSS and JavaScript that allows users to add new tasks, mark tasks as completed, and delete tasks dynamically using DOM manipulation.

2. SYSTEM REQUIREMENTS

Hardware Requirements:

- Computer or Laptop
- Minimum 4 GB RAM
- Keyboard and Mouse

Software Requirements:

- Operating System: Windows / Linux
 - Code Editor: Visual Studio Code / Notepad
 - Web Browser: Google Chrome / Microsoft Edge / Mozilla Firefox
 - Technologies Used: HTML5, CSS3 and JavaScript
-

3. ARCHITECTURE / DESIGN

The To-Do List application is designed using HTML, CSS and JavaScript.

HTML:

HTML is used to create the webpage structure, input field, Add Task button and task list.

CSS:

CSS is used to style the webpage, input field, buttons and task elements. Flexbox is used for proper alignment.

JavaScript:

JavaScript provides the main functionality of the application. The `addTask()` function adds new tasks,

`completeTask()` marks tasks as completed, and `deleteTask()` removes tasks from the list. DOM methods are used to create and modify webpage elements dynamically.

4. SOURCE CODE

To-Do List Application

```
<!DOCTYPE html><html lang="en"><head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>My To-Do List</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin: 0;
      padding-top: 10px;
    }

    h1 {
      font-size: 48px;
      margin-bottom: 40px;
    }

    .input-section {
      display: flex;
      justify-content: center;
      align-items: center;
      gap: 14px;
      margin-bottom: 35px;
    }

    #taskInput {
      width: 330px;
      padding: 14px;
      font-size: 20px;
      box-sizing: border-box;
    }

    button {
      padding: 12px 16px;
      font-size: 20px;
      cursor: pointer;
    }
```

```

#taskList {
    width: 600px;
    margin: auto;
}

.task {
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 8px;
    margin: 28px 0;
    font-size: 24px;
}

.task-text {
    min-width: 200px;
    text-align: right;
}

.completed {
    text-decoration: line-through;
}
</style></head>
<body>

<h1>My To-Do List</h1>

<div class="input-section">
    <input type="text" id="taskInput" placeholder="Enter a task">
    <button onclick="addTask()">Add Task</button>
</div>

<div id="taskList">

    <div class="task">
        <span class="task-text">lab record</span>
        <button onclick="completeTask(this)">Complete</button>
        <button onclick="deleteTask(this)">Delete</button>
    </div>

    <div class="task">
        <span class="task-text">assignment</span>
        <button onclick="completeTask(this)">Complete</button>
        <button onclick="deleteTask(this)">Delete</button>
    </div>

    <div class="task">

```

```

    <span class="task-text">coding pratice</span>
    <button onclick="completeTask(this)">Complete</button>
    <button onclick="deleteTask(this)">Delete</button>
</div>

<div class="task">
    <span class="task-text">practice code tantra problems</span>
    <button onclick="completeTask(this)">Complete</button>
    <button onclick="deleteTask(this)">Delete</button>
</div>

</div>

<script>
    function addTask() {
        let input = document.getElementById("taskInput");
        let taskText = input.value.trim();

        if (taskText === "") {
            alert("Please enter a task!");
            return;
        }

        let task = document.createElement("div");
        task.className = "task";

        task.innerHTML = `
            <span class="task-text">${taskText}</span>
            <button onclick="completeTask(this)">Complete</button>
            <button onclick="deleteTask(this)">Delete</button>
        `;

        document.getElementById("taskList").appendChild(task);

        input.value = "";
    }

    function completeTask(button) {
        let task = button.parentElement;
        let text = task.querySelector(".task-text");

        text.classList.toggle("completed");
    }

    function deleteTask(button) {
        let task = button.parentElement;
        task.remove();
    }

```

```
</script>  
</body></html>
```

5. COMPILATION / EXECUTION STEPS

1. Open Visual Studio Code or any suitable text editor.
 2. Create a new file named `todo.html`.
 3. Enter the above HTML, CSS and JavaScript code.
 4. Save the file with the `.html` extension.
 5. Open the `todo.html` file in a web browser.
 6. Enter a task in the input field.
 7. Click the **Add Task** button to add the task.
 8. Click the **Complete** button to mark a task as completed.
 9. Click the **Delete** button to remove a task.
 10. If the input field is empty, an alert message is displayed.
-

6. EXPECTED OUTPUT / SCREENSHOT

The final output displays the **My To-Do List** webpage with an input field, Add Task button, predefined tasks, Complete buttons and Delete buttons. Users can add new tasks, mark tasks as completed, and delete tasks dynamically.

Screenshot – Final Output:

My To-Do List

Enter a task	Add Task	
lab record	Complete	Delete
assignment	Complete	Delete
coding pratice	Complete	Delete
practice code tantra problems	Complete	Delete

7. RESULT / CONCLUSION

Thus, the **To-Do List application** was successfully developed using HTML, CSS and JavaScript. The application allows users to add, complete and delete tasks dynamically. This experiment demonstrates the practical use of JavaScript functions, event handling and DOM manipulation for creating an interactive web application.